La base de la tablet

Trabajo Práctico N.º 8 - Food Store en la feria

Equipo: Exequiel Arancibia / Joaquin Romero. Resolución sobre el Food Store acumulado de los trabajos anteriores. Se trabajó con copias privadas y productos de feria inventados; la fuente oficial foodstore se conservó.

1.a - Qué datos viajan y quién manda

Datos	Decisión y motivo
Producto: ID, nombre, precio, stock, disponibilidad, baja y fecha de copia	Viajan como catálogo acotado de cuatro productos. El precio se convierte a centavos; el stock local es orientativo. PostgreSQL decide los valores oficiales.
Borrador, ítems y solicitud UUID	Se crean en SQLite. El usuario es sólo un ID numérico de prueba, sin nombre ni contacto; la forma de pago es una opción, no datos de tarjeta.
Usuarios, contraseñas, mails, teléfonos; ventas confirmadas e historia completa	No viajan. No son necesarios para preparar el borrador y aumentan el riesgo si se pierde la tablet.

1.b - Esquema local y correspondencia

PostgreSQL 17.11; Python 3.12.14 con SQLite 3.53.1. El esquema empieza con PRAGMA foreign_keys = ON; se vuelve a activar y verificar en cada conexión. Cuatro tablas principales STRICT y una auxiliar de importación:

Tabla	Claves y reglas relevantes
producto_cache	producto_id conserva la identidad del servidor; precio_centavos/stock_visto enteros no negativos; flags 0/1; capturado_en.
pedido_borrador	id_local UUID textual PK; usuario_id_servidor; forma_pago sólo EFECTIVO, TARJETA o TRANSFERENCIA; fecha.
item_borrador	PK(id_local,producto_id), FK a borrador y caché; cantidad>0; precio_centavos_visto>=0.
cola_sync	solicitud_uuid PK; id_local FK al borrador; cuatro estados válidos; motivo. Índice único por id_local sólo WHERE estado=PENDIENTE.

El índice parcial admite otro intento histórico después de un conflicto, pero impide dos pendientes simultáneos. UUID de borrador y UUID de solicitud cumplen funciones distintas. La tabla snapshot_servidor contiene la instantánea nueva contra la que se compara.

1.c - Atomicidad, consulta offline y datos inválidos

Se ejecutó pruebas_sqlite.sql sentencia por sentencia. COMMIT conservó encabezado, dos ítems y solicitud; ROLLBACK de otra operación dejó cero filas. C1 consultó el catálogo local sin comandos PostgreSQL. C7 produjo SQLITE_CONSTRAINT_CHECK por cantidad=0 y SQLITE_CONSTRAINT_FOREIGNKEY por producto inexistente; cada ROLLBACK quitó también el encabezado previo.

Un error de sentencia no revierte por sí solo toda la transacción: ese rollback explícito es parte de la solución. Archivos y salidas completas: movil/*.sql y docs/evidencias/reproduccion_u8.json.

Instantánea y semáforo

2.a - Del servidor real al catálogo de la feria

Se creó foodstore_u8_arancibia_romero desde foodstore, con respaldo custom antes del fixture. Los IDs de la feria fueron asignados por identidad: 50001-50004. En el modelo heredado no existe producto.disponible: se deriva de stock>0. La baja de categoría también se transmite.

```
SELECT p.id AS producto_id, p.nombre,
       (p.precio*100)::bigint AS precio_centavos, p.stock,
       (p.stock>0) AS disponible,
       (p.eliminado OR c.eliminado) AS eliminado,
       now() AS capturado_en
FROM public.producto p
JOIN public.categoria c ON c.id=p.categoria_id
WHERE c.nombre='U8-FERIA-Arancibia-Romero';
```

BIGINT evita el límite de INTEGER32 al convertir NUMERIC(12,2). La exportación real se hizo con psql \copy a CSV HEADER. El importador valida t/f, los convierte a 1/0 y realiza UPSERT, sin REPLACE ni borrado de la caché referenciada.

Producto/ID	Centavos finales	Stock visto	Baja
Muzzarella / 50001	910000	10	0
Empanadas / 50002	120000	4	0
Fugazzeta / 50003	800000	5	0
Agua / 50004	150000	20	1

Instantánea final: 10/11/2026 12:32:46.308858 UTC-03, hora del servidor. Se conservan snapshot_inicial.csv, snapshot_c2.csv y snapshot_c4.csv; el último es movil/servidor_snapshot.csv. La lápida de Agua queda como fila, pero no aparece en el catálogo.

2.b - Decisión local, con prioridad roja

Prioridad	Estado	Regla
1 - Rojo	RECHAZADO	Ausente por LEFT JOIN, baja, indisponible, cantidad mayor al stock o borrador vacío.
2 - Amarillo	CONFLICTO	No hay rojo y algún precio actual difiere del visto.
3 - Verde	SIMULADO_APROBADO	No hay rojo ni amarillo. Es simulación; no confirma ni reserva una venta.

semaforo.sql sólo usa tablas SQLite y sólo persiste cambios sobre PENDIENTE. Se evalúa el lote; la primera ejecución puede resolver varios casos y las siguientes consultan los resultados ya guardados. La fecha local es diagnóstica y no decide autoridad.

2.c - C2: el cliente acepta un precio nuevo

```
-- Preparación administrativa, exclusivamente en la copia:
UPDATE public.producto p SET precio=precio+100.00
FROM public.categoria c WHERE p.categoria_id=c.id
AND c.nombre='U8-FERIA-Arancibia-Romero'
AND p.nombre='U8 - Muzzarella';
```

El runner acota además la categoría e impone que afecte una fila. Después exporta otra instantánea: visto900000 y servidor910000 centavos producen CONFLICTO. Se simula la aceptación, se actualiza el precio visto y se inserta un UUID nuevo: SIMULADO_APROBADO. El UUID anterior permanece CONFLICTO. La aceptación es ficticia de laboratorio, no una aprobación humana real.

Casos y conservación

Caso	Comando o preparación	Resultado real y explicación
C3	Borrador: Empanadas cantidad5; snapshot stock4. Ejecutar semaforo.sql.	RECHAZADO: solicitado=5, disponible=4. No se descontó stock.
C4	UPDATE de baja lógica de Agua en la copia; exportar e importar CSV nuevo.	RECHAZADO: dado de baja. Caché conserva eliminado=1 y catálogo visible devuelve cero filas para ese ID.
C5	Borrador Fugazzeta cantidad1, precio visto800000 y stock5, sin cambios.	SIMULADO_APROBADO. Conserva la etiqueta de simulación y no crea pedido del servidor.
C6	Reinsertar solicitud_uuid ya evaluado; después repetir semaforo.sql.	Error SQLITE_CONSTRAINT_PRIMARYKEY. La repetición dejó idénticos estado, motivo y evaluado_en. C2 conserva sus dos UUID.

Los comandos íntegros y parámetros están en tools/reproducir.py y evidencias/reproduccion_u8.json. Las escrituras PostgreSQL C2 (precio) y C4 (baja) son preparaciones administrativas explícitas, separadas de la fase puramente local.

Bordes y cierre de SQLite

También pasaron siete bordes: segundo pendiente rechazado por UNIQUE parcial; mezcla de rojo y precio cambiado; borrador vacío; producto ausente del snapshot; flag no disponible; stock cero; y rojo con precio cambiado en el mismo ítem. Los cuatro últimos usan copias efímeras en memoria, claramente diferenciadas de los CSV del servidor.

Cierre: integrity_check=ok; foreign_key_check sin filas; foreign_keys=1. La cola final tiene nueve solicitudes: 2 CONFLICTO, 4 RECHAZADO y 3 SIMULADO_APROBADO. En producción, la deduplicación definitiva iría en PostgreSQL: UUID único y creación de venta dentro de la misma transacción; no se implementa una venta en este TP.

Consulta de conservación en PostgreSQL

```
SELECT count(*) FROM public.pedido;
SELECT sum(stock::bigint) FROM public.producto;
-- 20_validar_no_ventas.sql agrega huellas de pedido/detalle
-- y de la secuencia ordenada (producto_id,stock).
```

Control de la copia	Antes	Después
Cantidad de pedidos	225000	225000
Stock global (50.004 productos)	5000038	5000038
Stock de cuatro productos de feria	10,4,5,20	10,4,5,20
Huellas completas de pedido y detalle	baseline	iguales

La fuente conserva además sus cinco huellas agregadas: 2 categorías, 50.000 productos, 20.001 usuarios, 225.000 pedidos y 499.997 detalles. No se exportaron filas de usuario. La huella completa de productos fixture sí cambia por las preparaciones C2/C4; el resto estable y el stock coinciden.

Segunda ejecución desde cero

Se conservó la primera copia como foodstore_u8_run01_conservada; con los mismos archivos se crearon otra copia canónica y SQLite vacía. Siete casos y siete bordes dieron la misma semántica; las cinco huellas fuente coincidieron entre ambas corridas. UUID, horas y bytes de las SQLite pueden diferir. La base entregada es la primera, sin cambios posteriores.

Estas comparaciones prueban estados observados de tablas; no auditan todos los cambios transitorios, secuencias o el clúster completo. No se usa la ausencia de logs como única prueba de no escritura.

Integración y mantenimiento

3.a - Tres motores, una autoridad de negocio

Motor	Qué conserva	Autoridad
PostgreSQL - U1 a U6	Catálogo, usuarios, pedidos y detalles; stock y precios oficiales; reglas, índices, vistas y seguridad.	Único que valida y confirma una venta real.
MongoDB - U7	Subconjunto experimental de pedidos1-5 y siete líneas, precios históricos Decimal128. Consultas contrastadas con PostgreSQL.	Documentos de la prueba; no manda sobre stock ni ventas. No se promovió a fuente oficial.
SQLite - U8	Catálogo copiado, borradores y cola con UUID; funciona offline.	Prepara propuestas. Stock y precio vistos pueden estar desactualizados.

3.b - Extensión del plan de Unidad 6

Qué se respalda	Cómo se prueba / quién
PostgreSQL: custom diario y antes de cambios; custodia privada.	Administrador: restauración semanal aislada, conteos/huellas y controles de reglas, índices y permisos. U6 observó RTO11,350s; su objetivo<=120s no es garantía universal.
SQLite: backup consistente al cerrar turno y antes de migraciones.	Encargado/soporte: restaurar en otra ruta, integrity_check, foreign_key_check y conteos de borradores/UUID/cola. Catálogo reconstruible.
MongoDB U7: fuentes y EJSON del subconjunto; backup si se usa operativamente.	Administrador: reconstruir colección aislada y repetir conteos/IDs equivalentes frente a PostgreSQL.

Antes de U8 se repitió el backup probado de U6: SUCCESS, 8.224.005 bytes; hash/manifiesto en evidencia. Cada laboratorio U8 produjo además su backup custom privado antes del fixture. No se incluyen dumps de usuarios en el ZIP.

Qué hacen las tablets tras una restauración vieja

Se invalidan TODAS las cachés e instantáneas y se descarga un catálogo completo nuevo, incluidas lápidas. Se conservan borradores y colas como evidencia; se reconcilian UUID y resultados con el servidor antes del reenvío. No se reutiliza un verde anterior ni se decide por la hora del teléfono o un umbral de antigüedad.

Durante la feria, el encargado revisa conflictos y rechazos al recuperar conexión y cada15min, y al cierre del turno. Soporte/administrador trata diariamente los no resueltos. Un precio nuevo requiere aceptación; falta de stock o baja requiere corregir el borrador, sin forzar valores del servidor.

4.a - Especificar antes de generar

Requisito EARS de Kiro	Caso que lo prueba
REQ01: integridad y persistencia offline atómica	C1 y C7; COMMIT/ROLLBACK
REQ02: semáforo con prioridad roja y aceptación de precio	C2-C5, mezcla y vacío
REQ03: UUID idempotente y una sola pendiente	C6 e índice parcial
REQ04: instantánea, lápidas y autoridad PostgreSQL	C4 y comparación de conservación

Se conservaron requirements.md, design.md y tasks.md reales de Kiro antes de OpenCode Build. Codex guardó por Copy Message As Markdown/Save As y extrajo los bloques sin reescritura; normalizó sólo finales de línea. El primer borrador y sus errores permanecen para trazabilidad. El commit ae17db49 precede a la generación original bcee3624.

Review: pruebas y decisiones

3.c - OpenCode Plan revisó todas las fuentes por partes: 83 anteriores y 7 finales U8, sin herramientas de ejecución. Los adjuntos grandes se dividieron sin omitir líneas. Se conservaron propuestas y respuestas; la disposición técnica final de Codex consolida 43 grupos y 64 referencias originales, todas con decisión. No se atribuye una revisión humana no observada.

Tres contrastes desarrollados en el motor

RH01 / TP1-TP5: se intentó DELETE del único detalle de un pedido final como app_web. El servidor lo rechazó con SQLSTATE 23514; ROLLBACK y cinco huellas de tablas iguales. Decisión: DESCARTADO como defecto del acumulado actual; la guarda heredada TP2 ya lo impide.

RH02 / integridad histórica y ACL U5: UPDATE pedido SET total=total+1 como app_web fue rechazado con SQLSTATE 42501. Se acepta el riesgo estructural para un escritor privilegiado, conservando la mitigación efectiva U5; no se lo presenta como vulnerabilidad de la app. La tabla quedó intacta.

RH04 / TP2: un INSERT CONFIRMADO sin líneas como administrador sí se aceptaba y se revirtió. Se agregó 30_corregir_estado_inicial.sql sólo en las copias U8: ahora rechaza con SQLSTATE 23514 y permite PENDIENTE con administrador bootstrap y app, siempre probado con ROLLBACK. Se conserva el trigger anterior de transiciones. Decisión: CORREGIDO. La segunda copia canónica final también tiene la guarda.

Comandos, rol, SQLSTATE y huellas: evidencias/review_bd_antes_correccion.json, review_bd_despues_correccion*.json. Un INSERT con identidad puede avanzar la secuencia de la copia aunque se revierta; no se afirma que la secuencia quede igual.

Tabla completa por grupos - antecedentes y Unidad 4

Grupo	Hallazgo u observación	Decisión
RH01	Borrado del último detalle de un pedido cerrado en esquemas históricos	descartado
RH02	Total almacenado escribible directamente	aceptado
RH03	Hipótesis de total perdido por concurrencia en esquema base	aceptado
RH04	Inserción directa de CONFIRMADO/ENTREGADO sin líneas	corregido
RH05	Orden de seed y restricciones estrictas	descartado
RH06	Cobertura de índice Q3 incompleta para id	aceptado
RH07	Alcance de mediciones antes/después de una corrida	aceptado
RH08	Comentario de equivalencia: contadores cero, no cero filas	aceptado
RH09	Índice adicional por producto_id como oportunidad de medición	aceptado
RH10	Instalación de restricciones no atómica ante interrupción	aceptado
RH11	Agrupación mensual TIMESTAMPTZ dependiente de TimeZone	aceptado
RH12	Proyecciones históricas no plenamente cubiertas por índices	aceptado
RH13	Detención del invocador psql ante errores de reconstrucción	aceptado
RH14	Aserciones de verificadores eliminables con Python optimizado	aceptado
RH15	Carga inicial confirmada antes de refresco que puede fallar	aceptado
RH16	Carga por rango fijo depende del siguiente valor de identidad	aceptado
RH17	Observación concurrente dependiente de pausa fija de dos segundos	aceptado
RH18	Matar cliente no confirma que su backend haya terminado	aceptado
RH19	Índices parcialmente solapados con distintos INCLUDE	aceptado
RH20	Excepción por profundidad de trigger admite otros orígenes anidados	descartado
RH21	Permisos del invocador para refrescar materializada desde un trigger	descartado
RH22	Consumo del reporte diario requiere conservar la zona de negocio	descartado

Agrupar no elimina referencias: review_hallazgos.md/.json incluye la correspondencia individual de las 64, sus fuentes, fundamentos y pruebas. “Aceptado” puede ser una recomendación condicional sin escenario ejecutado; no equivale a afirmar un incidente o una corrección inexistentes.

Review y declaración de IA

Grupo	Hallazgo u observación	Decisión
QA01	U5: secretos escapados en JSON	aceptado
QA02	U6: publicación y dump huérfano	aceptado
QA03	U6: diagnóstico local/nativo reducido	aceptado
QA04	U6: escaneo de prosa y adjuntos	aceptado
QA05	U5/U6: reintento con respuesta parcial	aceptado
QA06	U6: líneas no JSON en salida de OpenCode	aceptado
QA07	U7: residuo ante aceptación negativa inesperada	aceptado
QA08	U7: correspondencia fuentes/commit actual	descartado
QA09	U5: umbral 20000 en este dataset	descartado
QA10	U5: precondiciones de logging parciales	aceptado
QA11	U5: mezcla de artefactos entre corridas	aceptado
QA12	U5: membresía si una regresión autorizara GRANT	aceptado
QA13	U5: comprobar efecto del reset	aceptado
QA14	U5: salida de avisos NOTICE	aceptado
QA15	U5: marcado dentro de código del PDF	aceptado
QA16	U5: memoria total de objetos Git	aceptado
QA17	U5: rotación de log por segundo	aceptado
QA18	U5: falta archivo seguido en el worktree	aceptado
U801	U8: protección privada explícita del runtime	corregido
U802	U8: huellas completas frente a conteos	corregido
U803	U8: omitir un conflicto histórico artificial	corregido

QA01 se confirmó sólo con una cadena sintética y NO se marca como corregido el original U5. QA02 conserva la limitación de publicación, aunque el selector rechaza un huérfano. QA09 se contrastó en PostgreSQL: 20.000 IDs heredados 1..20000 y una cuenta 20001, huella heredada igual a U5. U8-01 fue una mejora de robustez; se matizó la afirmación de IA sobre mode=0700 en Windows: el soporte también se incorporó en Python 3.12.4.

4.b - Bitácora y Declaración de Uso de IA

La bitácora contiene ocho entradas: propuesta Codex; borrador Kiro; Kiro corregido; Build; Plan previo; revisión acumulada; dos laboratorios; Plan final y QA. Se registra qué se aceptó, corrigió o descartó. Prompts, respuestas fieles, adjuntos y hashes están en docs/ia/; la DUIA se entrega sin firmas simuladas.

Kiro generó texto; Codex lo guardó. OpenCode 1.18.23 (openai/gpt-6.1-sol) generó y revisó, sin ejecutar SQL; Codex hizo lectura técnica y pruebas locales antes de usar resultados. Sólo se compartieron contexto y fuentes saneadas autorizadas, no dumps, claves ni filas personales. El Plan final cubrió 7/7 y no identificó fallos actuales nuevos.

4.c - README y ensayo de diez minutos

README explica desde qué base acumulada se parte, cómo crear una carpeta sólo con fuentes, ejecutar desde cero, aplicar la guarda y conservar una copia antes de repetir. La rama review-final y los commits técnicos están en .git. Se entrega guion de 10 min con mapa, caso y hallazgo; el ensayo presencial y la defensa corresponden al equipo y no se afirman realizados por el agente.

Referencias técnicas y límites

Consigna TP8 y apunte U8 de la cátedra, paradas 1-10. SQLite: sqlite.org/foreignkeys.html, /stricttables.html y /lang_conflict.html. PostgreSQL 17: postgresql.org/docs/17/datatype-numeric.html. Python 3.12: docs.python.org/3.12/library/os.html#os.mkdir.

Revisión estática y ensayos acotados no son una auditoría universal. Se separan tablas, secuencias, copias y fuente; se conservan las evidencias completas y los riesgos condicionales. La plataforma recibe un ZIP; los respaldos completos y credenciales quedan privados.